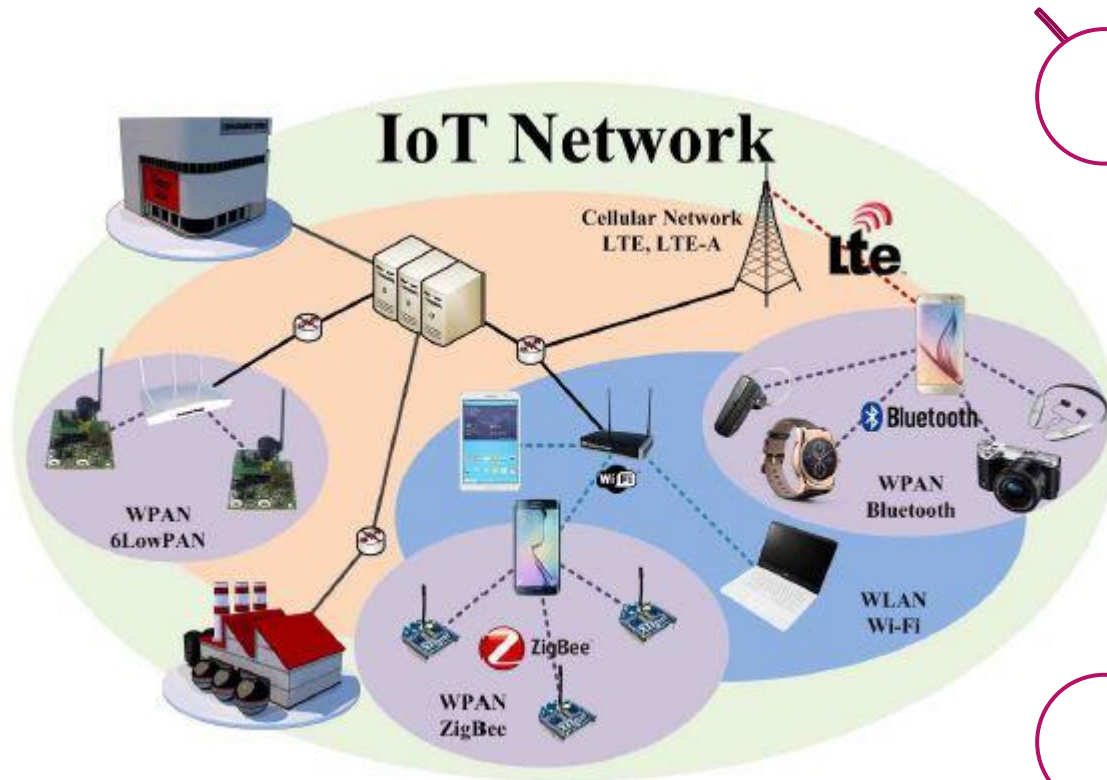


IoT Networking Landscape and Applications



IoT Networking Requirements

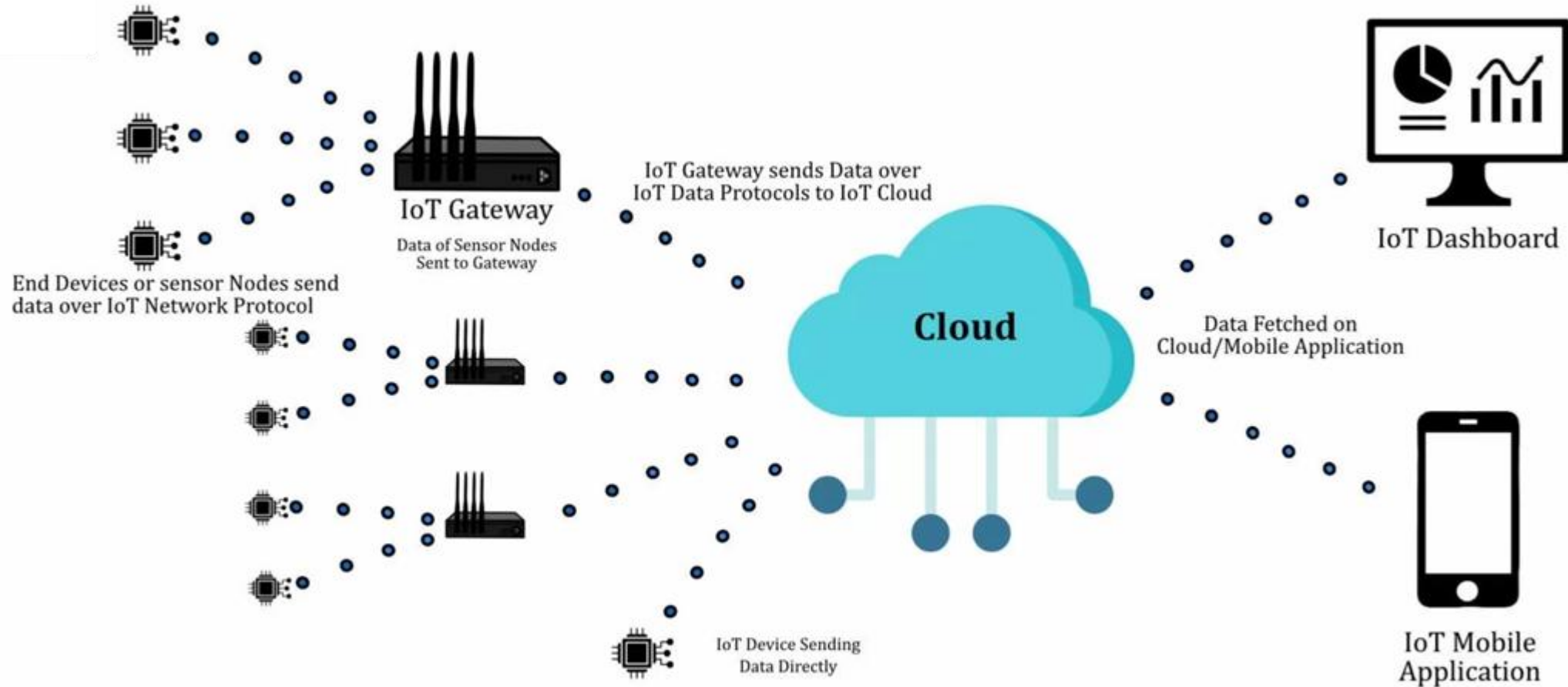
IoT Networking Elements

Protocol Landscape in IoT

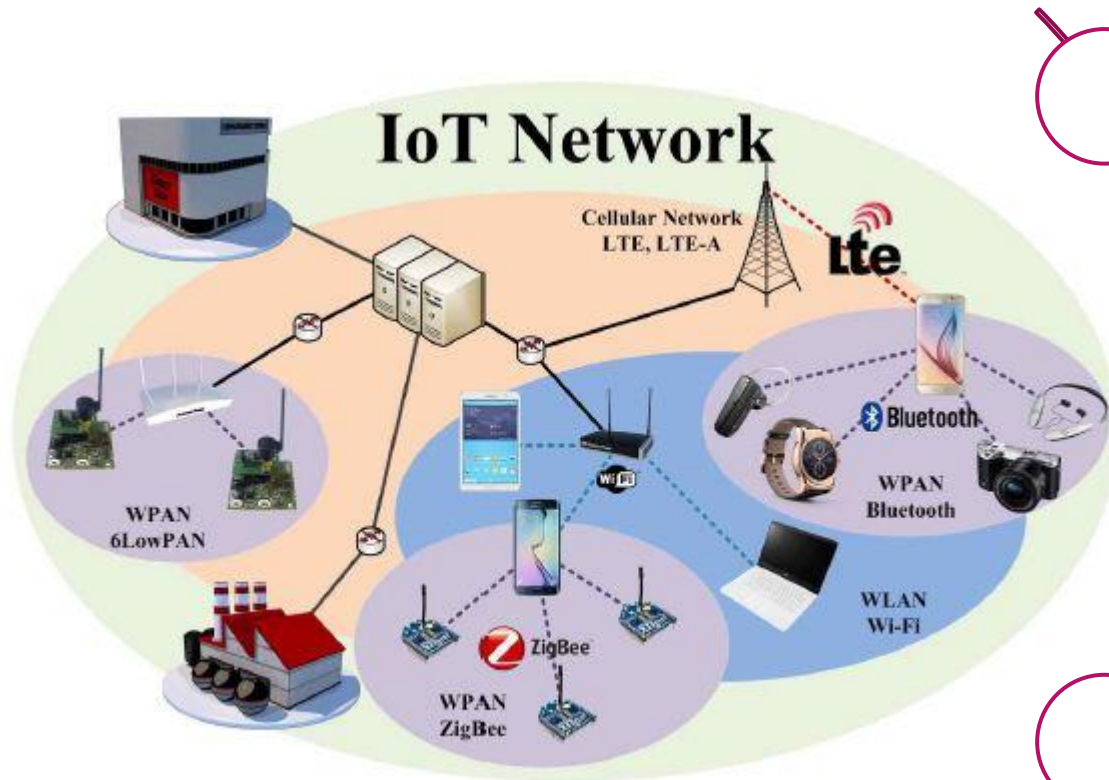
Comparison of various Protocols

MQTT Use case

Introduction to IoT Networking



IoT Networking Landscape and Applications



IoT Networking Requirements

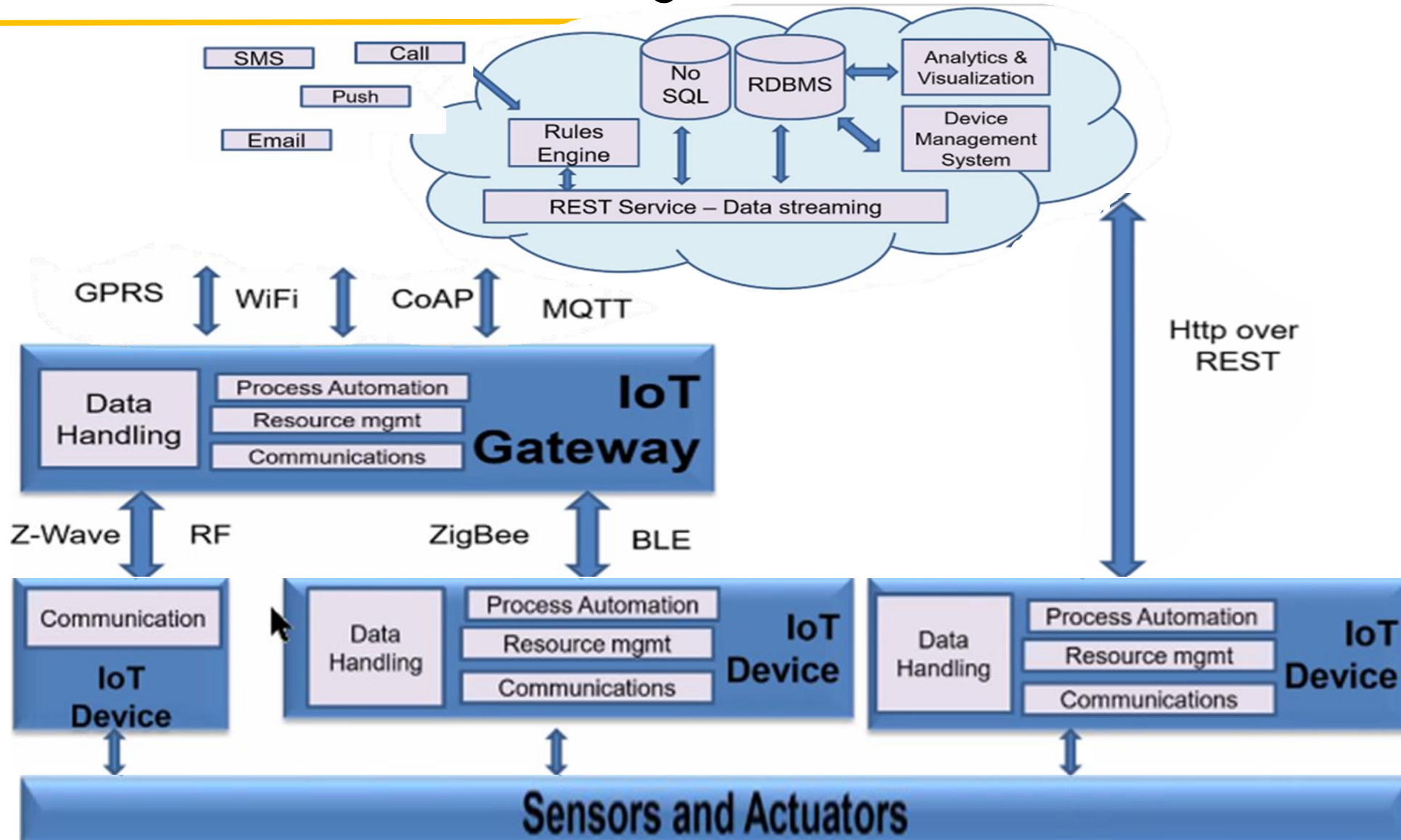
IoT Networking Elements

Protocol Landscape in IoT

Comparison of various Protocols

MQTT Use case

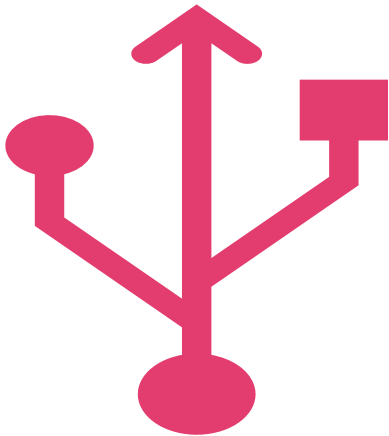
Introduction to IoT Networking



IoT Architecture

IoT Types of communication

Communication is the backbone of IoT—balancing speed, autonomy, and scalability.



Device-to-Device, or D2D communication

- Direct communication between IoT devices without intermediaries.
- *Example:* A temperature sensor signaling a cooling fan to turn on instantly.



Device-to-Gateway communication

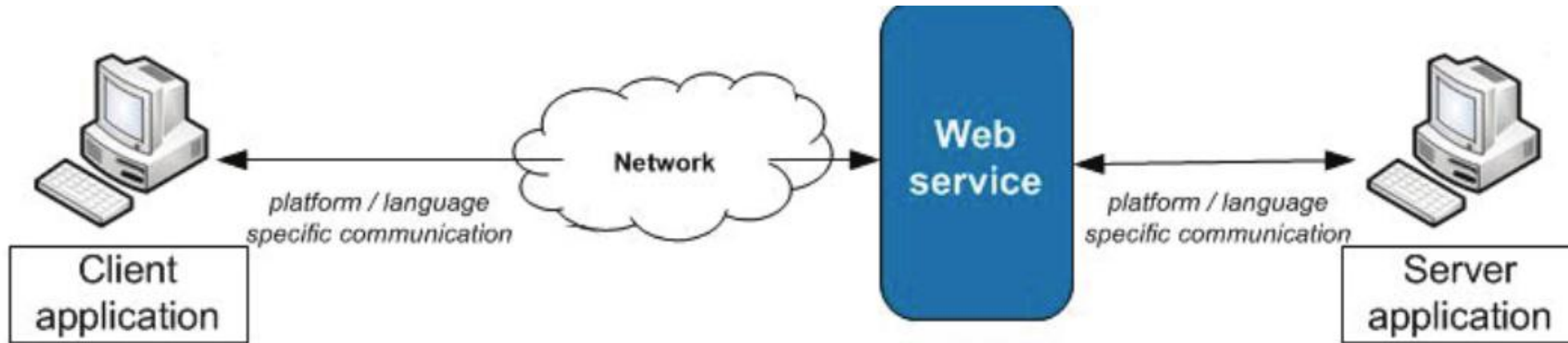
- Devices connect to an edge gateway for data aggregation, local processing, and secure transmission.
- *Example:* Industrial sensors sending data to an edge controller for real-time decisions.



Device-to-Cloud communication

- Devices communicate directly with cloud platforms for advanced analytics, visualization, and long-term storage.
- *Example:* Smart meters sending usage data to the cloud for billing and predictive analysis.

Networking requirement : IoT vs Web Apps



Homogeneous n/w stack – Default http / tcp / ip / ethernet

Substantial power availability - mostly do not have battery. Run on continuous power supply

Persistent connections - almost always available

Many to one connection - multiple client connecting to one server endpoint

Cost factor of Physical Layer - Not as high. Load is shared across multiple communication use cases

No real time requirement - large scala web application even work on eventual consistency

Client–Server Networking vs IoT Networking

Aspect	Traditional Client–Server Networking	IoT Networking
Network Assumption	Stable, reliable connection is assumed.	Connectivity is unpredictable, intermittent, or low-bandwidth.
Architecture Simplicity	Simple architecture — client ↔ server.	Multi-layer architecture — Device ↔ Gateway ↔ Edge ↔ Cloud.
Protocols Used	Mostly HTTP/HTTPS, REST APIs, WebSockets.	Multiple : BLE, Zigbee, Z-Wave, RF, Wi-Fi, LoRaWAN, NB-IoT, MQTT, CoAP, Cellular, Satellite , etc.
Communication Pattern	Request–response.	Multiple - Event-driven, publish–subscribe, streaming, real-time telemetry.
Bandwidth Needs	Typically, high bandwidth & stable networks.	Designed for low power, low bandwidth, lossy networks.
Scale of Nodes	Limited number of clients.	Massive scale — thousands to millions of devices.
Data Generation	Moderate, predictable data traffic.	Huge, continuous data streams from distributed sensors.
Power Dependency	Devices are always powered (PCs, servers).	Many devices run on battery; power efficiency is critical.
Mobility	Mostly stationary clients.	Devices may be mobile (vehicles, wearables, assets).
Environment	Indoor, controlled environments.	Harsh environments — outdoor, industrial, remote, forest, underwater.
Security Requirements	Standard cybersecurity controls.	Complex : device security, physical security, network security, data integrity, OTA updates.
Interoperability	High via standard web protocols.	Low interoperability — heterogeneous devices, sensors, platforms.
Failure Handling	Retry or refresh client page.	Must handle disconnections, buffering, store-and-forward, offline operation.

Client–Server Networking vs IoT Networking



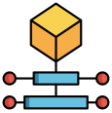
Networking : Client–Server assumes a stable and reliable connection

In IoT Connectivity is unpredictable, intermittent, or low-bandwidth



Architecture –Client–Server architecture is simple request response model —just a client talking to a server

IoT, on the other hand, is multi-layered : devices communicate with gateways, which connect to edge systems, and then to the cloud.



Protocols – Traditional systems mostly use HTTP or HTTPS, REST APIs, and WebSockets

IoT uses a wide range—Bluetooth Low Energy, Zigbee, Z-Wave, Wi-Fi, LoRaWAN, NB-IoT, MQTT, CoAP, cellular, even satellite.



Communication patterns – Client–Server is request–response

IoT is event-driven, often using publish–subscribe models, streaming, and real-time telemetry.

Client–Server Networking vs IoT Networking



Bandwidth – Traditional systems expect high bandwidth and stable networks.

IoT is designed for low power, low bandwidth, and lossy networks



Scale –Client–Server handles a limited number of clients

IoT scales to thousands—or even millions—of devices.



Data generation Traditional systems produce moderate, predictable traffic

IoT generates massive, continuous streams from distributed sensors.



Power dependency – PCs and servers are always powered.

IoT devices often run on batteries, so power efficiency is critical.

Client–Server Networking vs IoT Networking

	Client–Server Networking	IoT Networking
	Mobility – Traditional clients are mostly stationary.	IoT devices can be mobile—think vehicles, wearables, or assets.
	Environment –Client–Server operates in controlled indoor spaces.	IoT requires complex security—device-level, physical, network, data integrity, and over-the-air updates..
	Security : Traditional systems rely on standard cybersecurity measures.	IoT requires complex security—device-level, physical, network, data integrity, and over-the-air updates.
	Interoperability : Web protocols make traditional systems highly interoperable. .	IoT faces challenges with heterogeneous devices, sensors, platforms, and vendors.
	failure handling – Client–Server, you just retry or refresh	Client–Server, you just retry or refresh

IoT networking is far more complex and dynamic. It's not just about connectivity—it's about managing diversity, reliability, and scalability across massive ecosystems.